

Binary Tree: A binary tree is a tree data structure where each node has at most **two children**.

What is a Ternary Tree:

A Ternary Tree is a type of tree data structure where each node can have up to **three child nodes**.

N-ary trees:

The Generic trees are the N-ary trees which have the following properties:

1. Many children at every node.
2. The number of nodes for each node is not known in advance

Tree Traversal

Tree Traversal refers to the process of visiting or accessing each node of the tree exactly once in a certain order.

Depth First Search or DFS

Inorder Traversal

Preorder Traversal

Postorder Traversal

Inorder Traversal

Inorder traversal visits the node in the order: **Left -> Root -> Right**

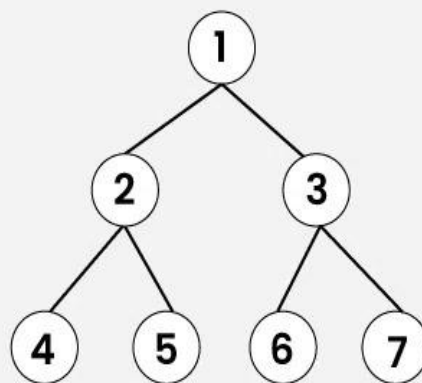
Preorder Traversal

Preorder traversal visits the node in the order: **Root -> Left -> Right**

Postorder Traversal

Postorder traversal visits the node in the order: **Left -> Right -> Root**

Tree Traversal Techniques



Inorder Traversal

4	2	5	1	6	3	7
---	---	---	---	---	---	---

Preorder Traversal

1	2	4	5	3	6	7
---	---	---	---	---	---	---

Postorder Traversal

4	5	2	6	7	3	1
---	---	---	---	---	---	---

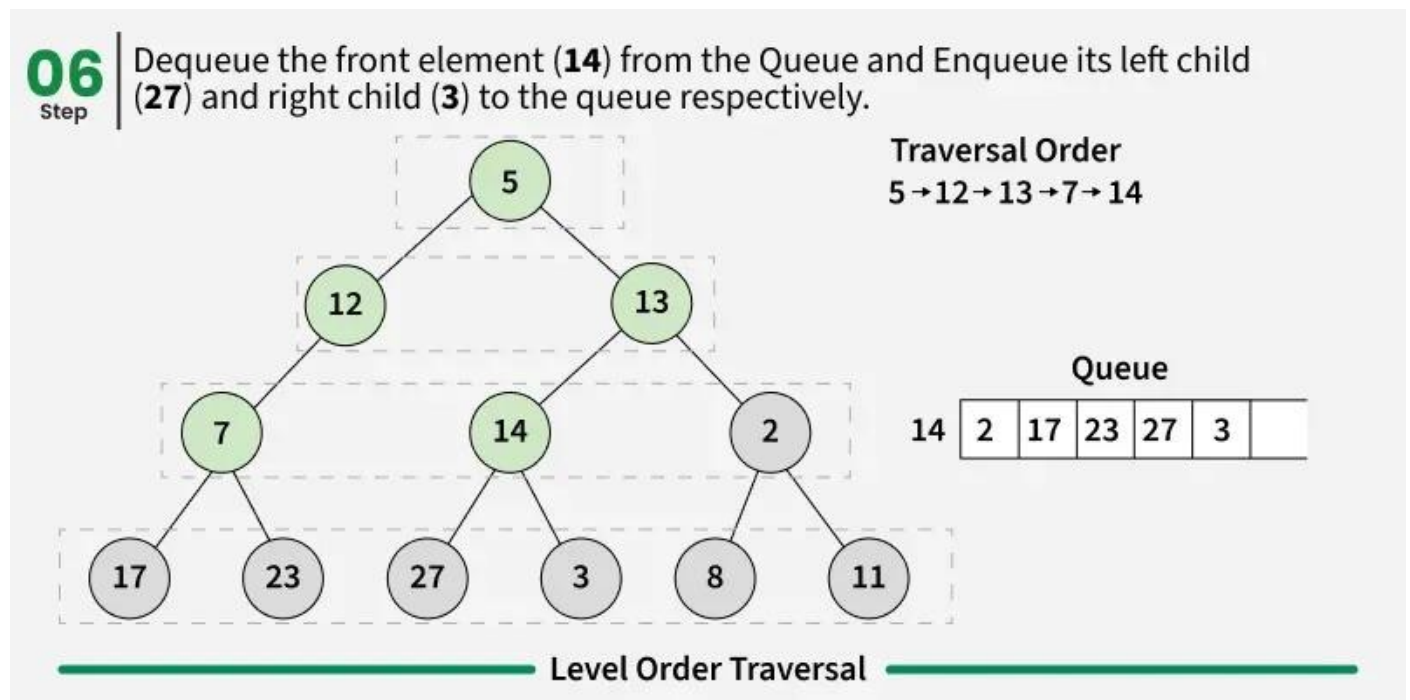
Level order Traversal

1	2	3	4	5	6	7
---	---	---	---	---	---	---



Level Order Traversal(BFS):

Level Order Traversal visits all nodes present in the same level completely before visiting the next level.

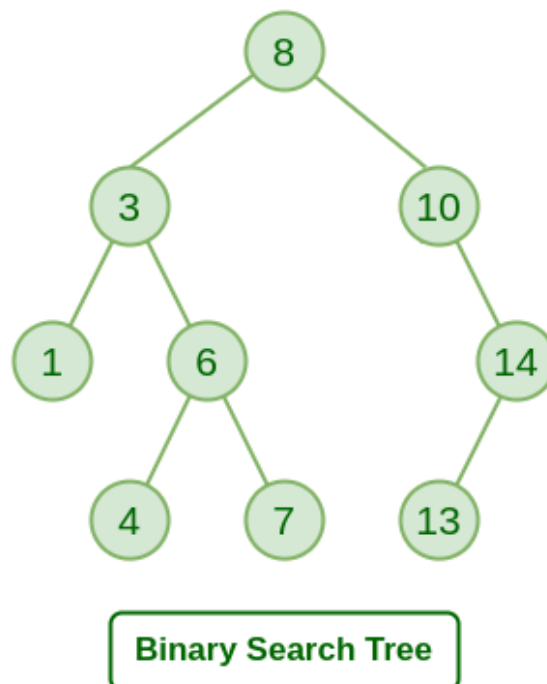


Binary Search Tree:

A **Binary Search Tree (BST)** is a type of binary tree data structure in which each node contains a unique key and satisfies a specific ordering property:

All nodes in the left subtree of a node contain values **strictly less** than the node's value.

All nodes in the right subtree of a node contain values **strictly greater** than the node's value.



Advantages of Binary Search Tree (BST):

- ❖ **Efficient searching:** $O(\log n)$ time complexity for searching with a self balancing BST
- ❖ **Ordered structure:** Elements are stored in sorted order, making it easy to find the next or previous element
- ❖ **Dynamic insertion and deletion:** Elements can be added or removed efficiently
- ❖ **Balanced structure:** Balanced BSTs maintain a logarithmic height, ensuring efficient operations
- ❖ **Doubly Ended Priority Queue:** In BSTs, we can maintain both maximum and minimum efficiently

Disadvantages of Binary Search Tree (BST):

- ❖ **Not self-balancing:** Unbalanced BSTs can lead to poor performance
- ❖ **Worst-case time complexity:** In the worst case, BSTs can have a linear time complexity for searching and insertion
- ❖ **Memory overhead:** BSTs require additional memory to store pointers to child nodes
- ❖ **Not suitable for large datasets:** BSTs can become inefficient for very large datasets
- ❖ **Limited functionality:** BSTs only support searching, insertion, and deletion operations